

# The Knowledge-Graph Build, Release and Serving System

Where a store is built, what a release is, how a package version and a store are paired, and how a serving host follows a release

Michael Volk

October 6, 2026

## Abstract

A knowledge-graph store is built on one host from the development LMDBs, under slurm, from a commit that carries a package tag; it is stamped with a release id and a **KgRelease** node, swapped into the serving container, and written out as an online backup that the archive on Taiga holds beside its manifest and checksums. A package version and a store are used only as a pair: the build refuses an untagged commit, the client refuses a store whose schema contracts the installed package does not reproduce for every dataset, the committed snapshots drive a pairs table that CI holds current, and **ops.sh sync** fails when two hosts serve different releases. The build host is GilaHyper today and is the one replaceable box in the diagram: the artifact it produces, the archive, and the deploy step that restores an artifact into any serving host do not depend on where the build ran. Radiant cannot serve a release yet, because Neo4j does not run a store on NFS and the VM has no other disk; the fix is a block volume and one **deploy** command.

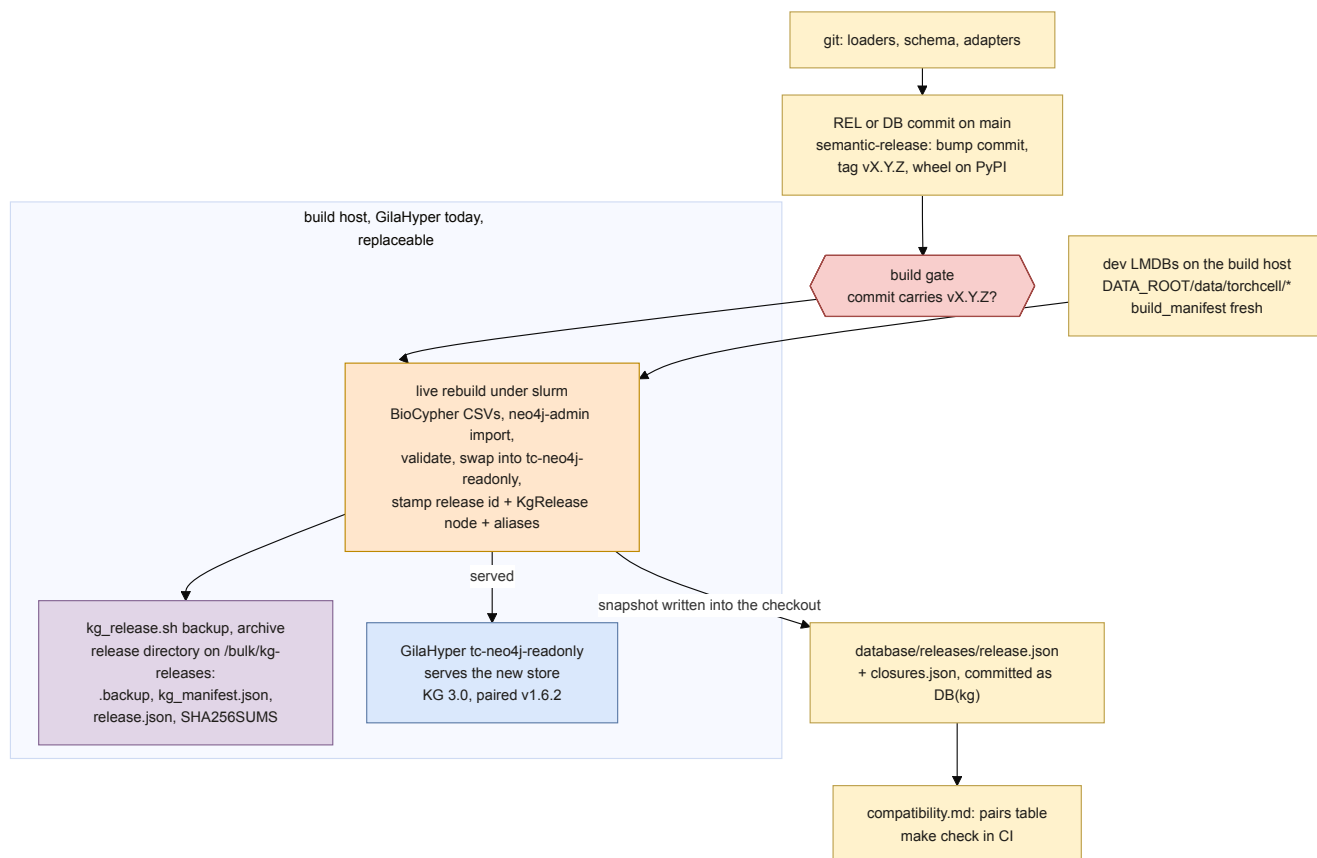
*Section status:* ✓ ready   ■ author-reviewed, keep stable   ✗ not done, agents may edit freely

*Provenance:* ▲ not in the library mirror; verify at the linked source   ● from a review's summary table, not the primary paper

## Contents

|   |                                     |   |
|---|-------------------------------------|---|
| 1 | The system in one figure ✗          | 2 |
| 2 | The build host ✗                    | 4 |
| 3 | Releases and pairing ✗              | 4 |
| 4 | The archive and the serving hosts ✗ | 6 |
| 5 | Recipes ✗                           | 6 |

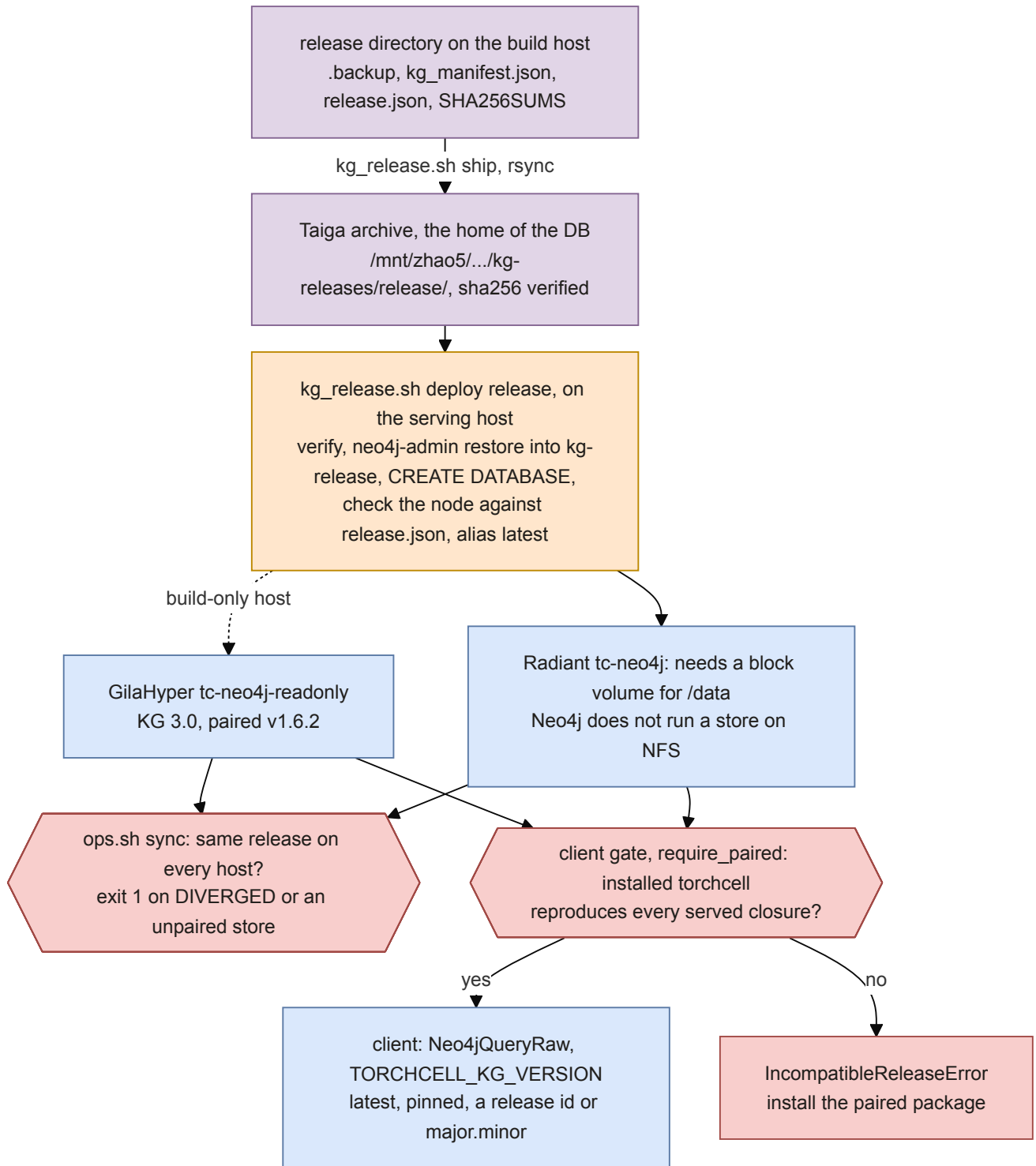
# 1 The system in one figure x



**Figure 1:** From source to release artifact. Yellow boxes are files in git or the development data tree; orange boxes are steps that run on the build host; the purple box is the release artifact; the blue box is the store the build host serves; red hexagons are gates that refuse rather than degrade. The cluster is the one box that moves when the build moves.

**Five stages.** A release passes through five stages, each with one owner and one output (Table 1). The source stage lives in git on every machine. The build stage is host-bound: it needs the development LMDB tree, docker, slurm and a fast local disk, and today that host is GilaHyper. The archive stage is Taiga, where a release is a directory of files and nothing runs. The serving stage is any host with a Neo4j container on local disk, and the client stage is whatever process opens a bolt connection with `torchcell` installed.

**What is host-independent.** Everything in Fig. 2. The backup file is Neo4j’s own online-backup format, the archive directory is plain files with checksums, and `kg_release.sh deploy` runs on whichever host has the container, reading the archive root from an environment variable. Moving the build changes the second column of Table 1 for one row.



**Figure 2:** From release artifact to a client. The archive on Taiga holds every release as verified files; **deploy** restores one into any serving host; **ops.sh sync** fails while hosts differ; the client gate refuses a store the installed package does not fully reproduce. Same palette as Fig. 1.

**Table 1:** The five stages, their owner, and the gate each one enforces.

| Stage   | Runs on                                           | Produces                                                                                                                                          | Gate                                                                                                      |
|---------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Source  | git, every machine                                | a tagged commit <code>vX.Y.Z</code> ; the committed snapshot<br><code>database/releases/&lt;release&gt;.json</code>                               | semantic-release bumps only on <code>REL</code> , <code>DB</code> , <code>PATCH</code> , <code>API</code> |
| Build   | the build host, under slurm                       | the store on <code>/db</code> , the <code>KgRelease</code> node, the manifest, the snapshot written into the checkout                             | the build commit must carry a package tag; every dev LMDB must be fresh                                   |
| Archive | the build host, then Taiga                        | <code>&lt;release&gt;/</code> with the <code>.backup</code> , <code>kg_manifest.json</code> , <code>release.json</code> , <code>SHA256SUMS</code> | <code>sha256sum -c</code> on arrival                                                                      |
| Serve   | any host with a Neo4j container on local disk     | database <code>kg-&lt;release&gt;</code> , aliases <code>latest</code> and <code>pinned</code>                                                    | the restored node must match <code>release.json</code> ; <code>ops.sh sync</code> fails on divergence     |
| Client  | any process with <code>torchcell</code> installed | an LMDB of query records                                                                                                                          | <code>require_paired</code> : every served dataset’s closure reproduced, else refused                     |

## 2 The build host ✗

**What the build needs.** The full build is `database/slurm/scripts/gilahyper_live_rebuild-slurm_docker.slurm`. It reads the development LMDBs read-only, checks that each one’s build manifest is fresh under the schema surface the container will install, generates the BioCypher CSVs in a build container pinned to the build commit, imports them with `neo4j-admin database import` into a new store beside the served one, validates it, swaps it under the serving container, and stamps it: the release id `<build date>-<commit[:8]>`, a `KgRelease` node carrying the version, the package version and tag, and one content hash per dataset, the aliases `latest` and `pinned`, and the committed snapshot. The old store stays beside the new one as `data.superseded.<timestamp>` for rollback. Table 2 lists what that script assumes of its host, which is the checklist for moving it.

**Table 2:** What the build host provides today, and whether a new host must match it.

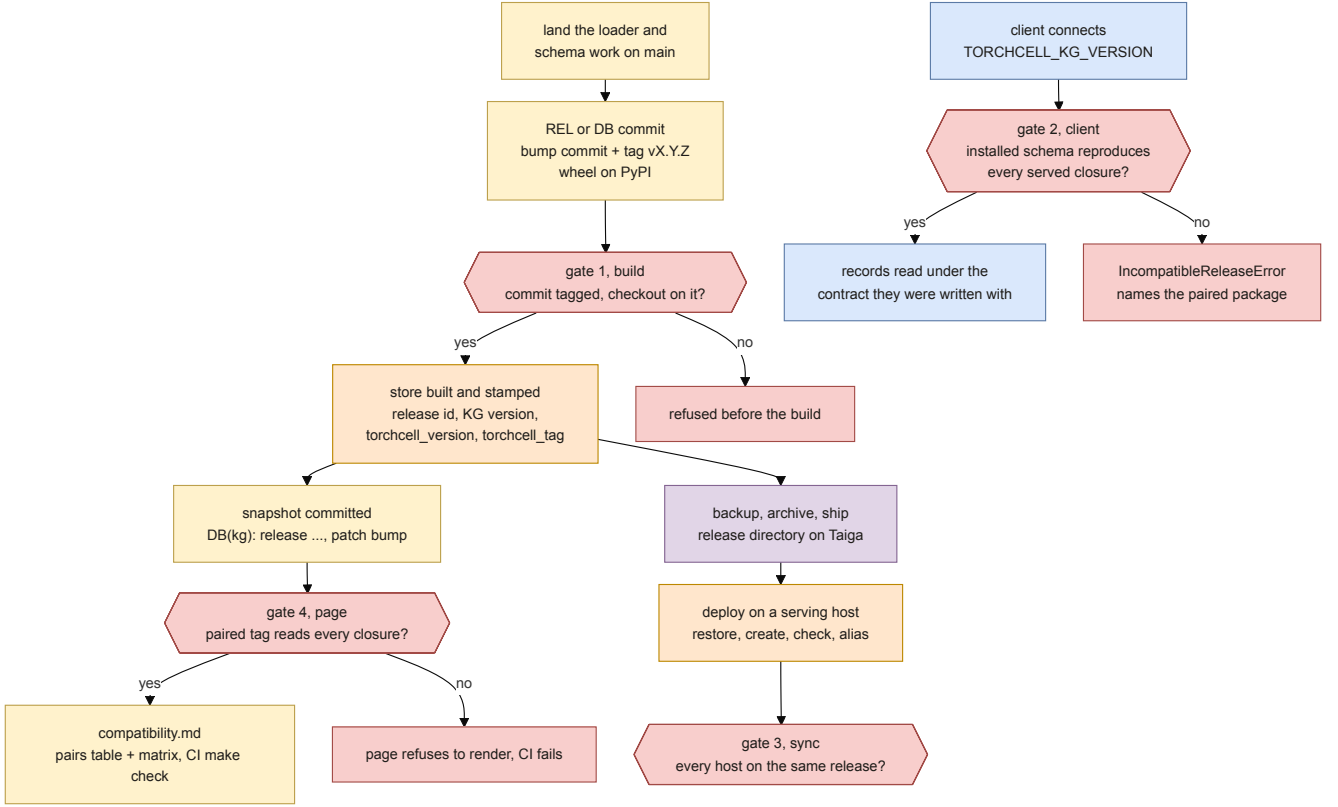
| Need                            | GilaHyper today                                                                                                                                                      | When the build moves                                                                                  |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Development LMDB tree           | <code>\$DATA_ROOT/data/torchcell/&lt;dataset&gt;/</code> on <code>/scratch</code> , one store per mapped dataset, each with a fresh <code>build_manifest.json</code> | must exist on the new host; the loaders rebuild it from the raw mirror, or it is copied               |
| Scheduler and container runtime | slurm with <code>-cpus-per-task=64 -mem=256G</code> ; docker with the <code>tc-neo4j</code> image                                                                    | any scheduler that runs one job with that budget; docker or an equivalent that honors the same mounts |
| Store disk                      | <code>/db</code> , 7.3 TB NVMe; the import writes the new store beside the served one                                                                                | local disk, not NFS; the 3.0 store is 151 GB restored, and a build holds two                          |
| Archive disk                    | <code>/bulk</code> , 24 TB RAID, mounted into the container at <code>/kg-releases</code> ; the online backup’s work directory holds the whole store uncompressed     | any local disk the container can mount; the work directory needs the store’s size again               |
| Fingerprint checkout            | the primary checkout at the build commit, which must be a tagged commit                                                                                              | the same; the tag gate is in the script                                                               |
| Serving container               | <code>tc-neo4j-readonly</code> , swapped in place with minutes of downtime                                                                                           | a build host need not serve; a build-only host ends at the artifact and ships it                      |

**What the last build measured.** Job 3297 built release `2026.10.06-4b293d34` (KG 3.0): 51 datasets, 100,127,515 nodes, CSV generation 3 h 09 min at 64 CPUs and 256 GB, the served store 151 GB on disk once restored. The online backup of that store is 11,175,064,969 bytes, and its copy to Taiga took 2 min 27 s at 111 MB/s. A restore of that artifact into a second database on the same DBMS, with serving uninterrupted, took 10 min 13 s (jobs 3338 and 3339).

**Build-only versus build-and-serve.** GilaHyper does both, which is why the script ends with a swap. A build-only host would stop after validation, run `kg_release.sh backup`, `archive` and `ship`, and let every serving host, GilaHyper included, pick the release up with `deploy`. Nothing in the pairing depends on which of the two shapes the build host has: the snapshot, the node and the tag are written either way.

## 3 Releases and pairing ✗

**Why a pair.** Every served record is serialized under the schema contract of the commit that built it, and the snapshot records, per dataset, the fingerprints of that dataset’s schema closure. A package version reads a release correctly only when its schema surface reproduces every one of those fingerprints. The October 2026 builds showed what happens



**Figure 3:** The life of one release. A package tag is cut first, the build records it, the snapshot commit bumps the package again, and from then on every reader of the store is checked against the contracts the store was written with. The red hexagons are the four gates; each one refuses rather than degrades.

without a rule: both ran from an untagged `main`, their snapshot commits used a tag the release parser does not bump on, and the compatibility page then showed every published package as partially compatible with the served store while the client read it anyway. A pair is the rule that replaces "partially": a release names one package, and a package either reads a release in full or is refused.

#### The four gates.

1. **Build.** The slurm script refuses to start unless the build commit carries a `vX.Y.Z` tag and the fingerprint checkout sits on it, so the stamp records the tag. A release is therefore cut before a build: a REL (minor) or DB/PATCH (patch) commit on `main`, semantic-release's bump commit and tag, then the build from that commit.
2. **Snapshot.** The build writes `database/releases/<release>.json` and its closures file into the checkout; they are committed as DB(kg): ..., a patch bump, so the package line advances with the store.
3. **Client.** At connect, `Neo4jQueryRaw` reads the resolved database's `KgRelease` node and runs the pairing check (`require_paired` in `torchcell/knowledge_graphs/releases.py`) against the installed schema surface before opening a driver. Any drifted or unverified dataset, or a store with no release node, raises `IncompatibleReleaseError` naming the paired package and the install command. There is no override.
4. **Page.** `scripts/kg_compat_page.py` renders a pairs table and the tag-by-release matrix from the committed snapshots; a paired tag that stops reading its release is a broken pair and the generator refuses, which fails the `-check` step in CI.

**Table 3:** The pairs as of 2026-10-06. "Reads it" lists every published tag whose schema surface reproduces all 51 closures; the paired package is the one the release names.

| KG release          | KG version | paired package | reads it         |
|---------------------|------------|----------------|------------------|
| 2026.09.21-ab6d8c5d | 1.2        | v1.2.1         | v1.2.1 to v1.6.1 |
| 2026.10.02-833970cd | 2.0        | v1.6.1         | v1.2.1 to v1.6.1 |
| 2026.10.06-4b293d34 | 3.0        | v1.6.2         | v1.6.2           |

**Repairing a release built before the rule.** `releases retag -release <id> -tag vX.Y.Z` pairs a snapshot built from an untagged commit with a tag cut afterwards, and accepts the tag only when the schema surface at that tag reproduces every served closure. It is idempotent for the same tag and refuses a second, different tag. `kg_release.sh retag` runs it with the manifest and then rewrites the served node. That is how the three releases in Table 3 got their packages; from the next build on, the stamp records the tag and no repair is needed.

## 4 The archive and the serving hosts ✗

**Taiga is the archive, not a store.** A release directory holds the online backup, the manifest at archive time, the `KgRelease` node's properties as `release.json`, and `SHA256SUMS`. `kg_release.sh ship` copies it to `/mnt/zhao5/mjvolk3/projects/torchcell/kg-releases/<release>/` on Taiga and verifies the checksums there; `list` shows both copies; `keep` and `purge` manage retention. Taiga is where the database lives in the sense that matters for recovery: every release is a verified set of files that any host with Neo4j can restore. It is not where a database runs.

**Why Neo4j does not run a store on NFS.** Neo4j keeps its store as memory-mapped page files and coordinates access with file locks, and its operations manual lists NFS as an unsupported filesystem for the data directory [▲](#) [external: Neo4j operations manual, "File system requirements": NFS is not supported for the database store]. The failure is not theoretical. The Radiant VM's container serves its old 35-dataset store from the Taiga mount, and every cold page read raises `java.io.IOException`; `ops.sh status` shows the database online with 7,180,465 nodes and no readable release node. A restore onto NFS would reproduce that fault for the new store.

**The deploy step.** `kg_release.sh deploy <release> [-pin]` runs on the host that will serve. It pulls the release directory from Taiga when the local archive root lacks it, verifies `SHA256SUMS`, restores the artifact with `neo4j-admin database restore` into a new database `kg-<release>` while the DBMS stays online, brings the database up with `CREATE DATABASE ... WAIT`, compares the restored node's release id, version and dataset count with `release.json`, and only then retargets the `latest` alias (and `pinned` with `-pin`). The previous database stays for rollback. It ends with `ops.sh sync`, which exits 1 while two hosts serve different releases or any host serves a store without a release node. `restore-test <release>` is the same restore and the same checks into a throwaway database, dropped at the end; it is what proved the 3.0 artifact restorable (10 min 13 s, counts identical to the served store).

**The fix for DIVERGED.** Radiant has 16 CPUs, 62 GB of memory, a 40 GB root disk and the Taiga mount, and no other disk. Until it has a local block volume for the container's `/data` there is no Neo4j-side fix, and `ops.sh sync` stays DIVERGED by design. The fix is two steps.

1. Attach a block volume and mount it at the path the container maps to `/data`. The restored 3.0 store is 151 GB, so 500 GB holds two releases side by side with headroom; the earlier 2 TB estimate predates the store shrinking to this size.
2. On Radiant, with the archive root pointed at Taiga:

```
ARCHIVE_ROOT=/mnt/zhao5/mjvolk3/projects/torchcell/kg-releases \
NEO4J_CONTAINER=tc-neo4j \
  bash scripts/kg_release.sh deploy 2026.10.06-4b293d34 --pin
```

After that, every release is `ship` then `deploy`, and the served version on Radiant follows the archive. The one interim alternative is a bolt proxy on Radiant forwarding port 7687 to GilaHyper, which gives clients one hostname but keeps the live store on GilaHyper rather than Taiga; it has not been set up.

## 5 Recipes ✗

**Cut a release and build.** Land everything the store should serialize under. Push a REL commit for a minor, or DB/PATCH for a patch; semantic-release pushes the `vX.Y.Z` bump commit to `main` and publishes to PyPI. Fast-forward the build checkout to that commit. Confirm every mapped dev store reads fresh under `python -m torchcell.provenance.build_manifest`, then submit:

```
sbatch --cpus-per-task=64 --mem=256G --time=12:00:00 \
  database/slurm/scripts/gilahyper_live_rebuild-slurm_docker.slurm
```

The script refuses an untagged commit before touching anything. When it finishes, commit the snapshot it wrote as DB(kg): <release> snapshot ..., run `python scripts/kg_compat_page.py`, and commit the page.

### Archive and ship.

```
sbatch -J kg-release -c 4 --mem=16G --time=6:00:00 \
  --wrap "bash scripts/kg_release.sh backup"
# backup runs detached inside the container; wait for 'exit 0' in backup.log
sbatch -J kg-release-ship -c 2 --mem=8G --time=2:00:00 \
  --wrap "bash scripts/kg_release.sh archive && bash scripts/kg_release.sh ship"
```

ship ends with `sha256sum -c` on Taiga; the release is archived only when that prints OK for every file.

### Deploy, test a restore, repair a pairing.

```
bash scripts/kg_release.sh deploy <release> [--pin]    # on the serving host
bash scripts/kg_release.sh restore-test <release>      # throwaway database, dropped
bash scripts/kg_release.sh retag <release> vX.Y.Z      # untagged build, later tag
make ops-sync                                         # exit 1 on DIVERGED
```

On GilaHyper each of the first three runs under slurm with `sbatch -c 4 -mem=16G -wrap "..."`, since they touch the DBMS.

**Choose a release from code.** Set `TORCHCELL_KG_VERSION` to `latest` (the default), `pinned`, a release id, or a `major.minor` version. The client resolves it to a database, reads that database's release node, and refuses with `IncompatibleReleaseError` unless the installed `torchcell` reproduces every served closure. The error names the package to install.